HyMeKo, XML, YAML and JSON: equivalent-data parsing comparison

Created-at: 2026-09-07 20:33 +02:00

Owner and authority. Parser owns the frontend benchmark. User explicitly requested an exhaustive comparison with XML, YAML and JSON. Only benchmark/test dependencies and clients change; production parser and Core are unchanged. Parser HEAD equals freshly fetched origin/master 284c490, clean at task start. Core's earlier uncommitted optimizations remain untouched. The provider map and actual parser manifest confirm the owner. No local Parser CORE.YAML exists; no production dependency is added, and this request authorizes the required isolated test libraries.

Scope and ETA. Approximately 40 minutes: helpers/build/tests 12, production-scale smoke 3, two measured rounds 20, analysis/report 5. Five patterns (flat, linked, Unicode, long text, nested), four logical-node scales (100, 1000, 10000, 50000), four formats and three boundaries (serialize, parse document, decode common records): 240 cells per round, two rounds. Ten post-warmup samples per cell using Criterion flat sampling. This is a broad declared corpus, not proof for all inputs or all implementations of each language.

Representation and equivalence. One deterministic owned record tree: unique identifier, finite numeric value, text, signed weighted links and children. Generate each case once, serialize all formats, and compare every decoded record against the generator before timing. Fixture manifest records bytes, hashes, counts, depth and generation/encoding diagnostic times. Library-native AST/DOM representations differ and must be labelled. Decode returns the same owned records. This is not HyMeKo semantic IR compilation or execution; no unfair comparison against hashing or reference resolution. Both bytes/s and logical records/s accompany latency.

Libraries. Existing HyMeKo SIMD-dispatched parser; serde_json; roxmltree for XML DOM, quick-xml for XML serialization/typed decoding; serde-saphyr for YAML generic values/typed decoding. Record exact lockfile versions and settings. Keep default parser limits and report any rejected case. No CUDA, QEMU or Python parser timings mixed into this same-host Rust comparison.

Affected files. Parser Cargo.toml and Cargo.lock (dev dependencies and bench registration), benches/parser_bench.rs (assert successful parses), benches/format_bench.rs, benches/formats.rs (or a cohesive helper directory if needed), tests/format_comparison.rs, scripts/format_campaign.py, this four-format plan and reports/2026-09-07-format-comparison.md plus its numerical/plot evidence. Existing Core memory harness is reused, not copied; no production API change.

Tests and experiment. Round-trip all shapes and sizes before scheduling; tests independently assert record fields, signs/weights, Unicode/escapes, nesting, empty values, malformed/truncated sources and semantic mismatch detection. Original HyMeKo generators must parse successfully instead of timing an unchecked Result. Run Parser tests and applicable formatting/lint. Preflight new helpers. A maximum-size smoke precedes the matrix. Each parse/decode process loads only

its resident input; serialization processes load the common record tree. Generation and I/O are outside timers. Exclude final destruction from phase timers. Verify binary and fixture hashes.

Budgets and observability. At most 16 GiB observed peak resident memory and 180 seconds per cell, enforced by the campaign monitor; VAS is not capped. Fresh subprocess per cell, sequential on a quiet host. Capture growing logs, exit status, CPU/wall/progress and peak memory; resume only verified completed cells from the same binary and fixtures. Failure is recorded, never a speed result. Large strings/deep nesting can hit library limits; record the cause, do not silently shrink or disable guards. Report median, inclusive IQR, worst and all raw Criterion samples.

Output and rollback. A readable report, raw JSON and modern interactive plots plus an exportable figure. State library/phase, input semantics, sample count, platform and limitations on every plot. No universal format ranking. Revert only named benchmark changes if invalid; retain diagnostic evidence and user changes.

Correctness finding, 20:39. The independent mixed escape/Unicode round-trip fails on the actual parser: lex_string casts UTF-8 bytes to characters on its escape path. Extend the scope narrowly to src/lexer/common.rs: accumulate decoded bytes and validate UTF-8 once when constructing the owned string. Preserve the existing ASCII escape and unknown-escape behavior. No dependency or public API change. Freeze the failure and test scalar/SSE2/AVX2 before measuring; record the uncommitted correction in benchmark provenance. This repairs an observed correctness defect discovered by the requested comparison, not a speculative speed change.

Budget finding, 20:47. The YAML default rejects the flat 50k fixture at 250001 parser nodes (limit 250000). Record default-policy rejection per fixture, and use a separately disclosed finite corpus budget of 5 million YAML nodes and 10 million events for measured YAML cells. Keep all other YAML limits, including depth 64 and scalar bytes 64 MiB. Never time default rejections as successful parses. This explicit policy permits the same large workload without disabling guards.

Publication, 20:56. Reuse the existing Next.js Observatory and Plotly renderer/HUD on Amaterasu; add a Matched formats tab and a typed observation adapter. Affected web files: src/app/observatory.tsx, src/app/formats.tsx, src/lib/formats.ts, src/app/globals.css, tests/format.test.mjs, tests/browser.mjs, public/formats/*, tools/plot_reports.py and compose.yaml. The web checkout is local-only (no remotes), main caca3a5, with prior authorized UI changes preserved. Parser owns numerical aggregation in scripts/format_report.py; the web consumes verified artifacts only. Reuse its exporter for SVG. Validate same-source values, complete coverage, hover/pin, 2D/3D axes and mobile layout; then build and deploy with rollback to the existing purple-platforms image. ETA from measured initial cells: campaign completes around 21:10, report and browser-verified publication around 21:20. No runtime algorithm or Core change.

Stability follow-up, 21:10. All 480 cells succeeded in 836.6 seconds, but 53 of 240 paired observations differ by more than 20 percent between rounds. Balanced power and hybrid CPU scheduling are candidate explanations, not established causes. Add an optional affinity mask and shared-fixture path to the existing runner. Run nested/long-text 50k, all four formats and all three phases, two rounds (48 cells) on logical CPU 0. Preserve the original campaign and publish this as a separate sensitivity check; do not selectively replace slow values. Verify child affinity. Expected 2–3 minutes, same 16 GiB/180 s budgets; source/binary unchanged.

Browser finding, 21:25. A direct Matched formats load fetches data asynchronously; the shared HUD resolves its dock before that element exists and keeps a floating popup. Fix the existing ObservationProvider lifecycle to observe late dock insertion and detach the observer on

resolution/unmount. Add an optional dataset guide URL to Observation so format points link to their own guide. Let the existing Plotly renderer preserve a caller-supplied legend title. Additional web files: src/app/observation.tsx, src/lib/observations.ts, src/app/plot.tsx. Verify direct-load desktop docking, mobile floating fallback, cleanup and existing views.

Optimization continuation, Created-at: 2026-09-08 02:17 +02:00. User requests further optimization. ETA 40 minutes: symbolized profile/build 10, focused implementation and tests 15, matched before/after experiments and report 15. Parser owns this work; Core protected items: empty. Fresh Parser fetch matched origin/master 284c490; preserve all previous campaign edits. Reuse this plan, format benchmark, campaign runner and report; keep new evidence under reports/format-comparison/optimization. Before changes preserve source and matching executable/PDB. Profile actual native AST parsing on linked and long-text 50k fixtures with samply; no adapter costs attributed to Core. Candidate causes: grammar temporary list allocation, lexer repeated scans, generated parser stack movement. Select only a demonstrated hot path. Possible affected production files: src/hymeko.lalrpop and src/lexer/common.rs or simd.rs; no public API, accepted language, AST shape or dependency change. Extend tests/format_comparison.rs for boundaries, malformed input, long lists, nesting and SIMD parity. Extend existing runner only if needed for focused format selection. Unchanged fixtures/hashes, fixed CPU 0, identical release/debug settings for both binaries, at least two independent rounds of ten post-warmup samples; reverse A/B order and investigate spread above 10 percent. Smoke at 50k before focused matrix. Preserve 16 GiB RSS and 180-second process guard; latency/memory regression above 10 percent blocks acceptance until isolated. Profile includes destruction and harness: separate phase times remain the speed evidence. No claimed Core IR/runtime or cross-platform speedup without separate measurement. Roll back only this continuation's production hunks if no demonstrated benefit; keep original Unicode fix and baseline evidence. Emit raw values, median/IQR/worst, matching provenance, profile and comparison figure.

String scanning continuation, Created-at: 2026-09-08 02:50 +02:00. User authorizes the next measured Parser hotspot. ETA 25 minutes: implementation and semantic controls 8, paired measurements and profile 8, gates/report 9. Reuse final.exe/final-source.zip and the long-text CPU profile from the preceding optimization as baseline; do not discard the grammar optimization. Fresh fetch: Parser master and origin/master 284c490, zero divergence; previous dirty files kept. Owner: Parser. CORE.YAML items touched: empty. No new dependencies or public APIs. Affected files: src/lexer/common.rs, tests/format_comparison.rs, this plan bundle, existing format-comparison report, generated optimization/string-scan evidence. Add one private pure helper to skip ordinary string bytes using safe eight-byte loads and a word-parallel zero-byte test for quote/backslash; scalar search finds the exact delimiter and handles tails. Preserve source-borrowed strings, UTF-8 escape decoding, source spans, unterminated/invalid-UTF8 errors and lexer position on failure. No new unsafe code, CPU feature assumption, global state or allocation. Test every byte/lane and short tail against an independent scalar search, delimiter pairs and high-bit bytes; integration tests at block boundaries, long strings, Unicode, known/unknown escapes, malformed endings and all lexer tiers. Run on baseline and candidate before timing. The existing 50k long-text profile places lex_string in 26.07 percent of parse samples; this is measured, not conjecture. Smoke 50k then focused native-AST paired A/B for all five patterns, 100/10000/50000 records where needed, ten warm samples per process and two independent rounds with reversed case/variant order. Same binary flags and CPU 0 affinity; existing 16 GiB and 180-second process guards. A latency or repeated-memory regression over 10 percent requires investigation; discard an unhelpful candidate. Record sample spread, do not replace slow values. Profile final long-text path separately from timings. Preserve exact binaries/PDB/source, numerical and plotted output. Core IR/runtime and downstream deployment remain separate unmeasured boundaries. Rollback only this continuation's named changes; preserve the earlier UTF-8 repair and LALRPOP grammar improvements.

AST list continuation, Created-at: 2026-09-08 03:36 +02:00. ETA 25 minutes: bounded list change and semantic tests 8, paired JSON controls and profile 10, gates/report 7. Parser master 284c490 matches freshly fetched origin/master; preserve all earlier dirty changes. Owner Parser, protected Core items empty, no dependency or public type/API change. User requests JSON parity before continuing the original CUDA plan. Treat that condition as unfulfilled until matched parsing results support it; record DOM/AST and typed decoding separately and do not substitute a single favorable workload for the whole suite. Current long-text profile attributes 23.5 percent of parse samples to HyperItem vector growth. Generated HyperItem+ seeds vec![first], so every two-item body grows immediately. Hypotheses: tiny-body reallocations versus large-list growth. Introduce private HyperItems/HyperItemsMany grammar rules: empty and singleton keep exact allocation behavior; the first pair is allocated together, and later items append normally. This targets two-item bodies without oversizing singletons. No input pre-scan, guessed byte-to-node capacity, or parallel grammar/parser. Affected production file src/hymeko.lalrpop; extend tests/format_comparison.rs with empty/single/pair/large mixed node-edge-arc sequences and malformed endings. Extend benches/format_bench.rs only if an untimed capacity/shape diagnostic is needed. Reuse scripts/format_campaign.py and format_report.py; evidence in optimization/item-list. Preserve before/after binary, PDB and exact source. Smoke max-scale flat/linked/long-text before timed matrix: all five patterns, native AST and common records at 50k, two fresh rounds with ten samples, CPU 0, rotated baseline/candidate/JSON order. Check small and singleton-heavy sources separately for memory regressions; use existing fixture/semantic oracles. Every process retains 16 GiB observed working-set and 180-second guards. Investigate latency or memory regression over 10 percent. Capture candidate profile on long-text 50k, compare inclusive stacks without summing nested shares. If the targeted change fails, restore only this stage's production changes and retain evidence. No GPU work while latency samples run. CUDA follow-up reuses the existing Phase 6 feasibility and platform-audit plans when the user's parser condition is met; no new backend or dependency is implied by this step.

Remaining frontend costs, Created-at: 2026-09-08 04:00 +02:00. ETA 25 minutes: linked/Unicode profiles 5, isolated implementation and tests 8, paired three-way measurements 7, gates/report 5. Fresh Parser fetch matched master/origin 284c490 with existing changes preserved. Owner Parser; protected Core items empty; no dependency, public API or new unsafe code. Preserve current item-list executable/PDB and source as baseline under optimization/frontend. Profile linked and Unicode native AST at 50k before choosing changes. Distinguish grammar reduction/stack movement, numeric conversion, allocation and string escape decoding. Candidate files src/hymeko.lalrpop, src/lexer/common.rs or simd.rs; existing tests/format_comparison.rs must cover every touched path and preserve values, order, borrowing, Unicode and malformed-input diagnostics. No manual edits to generated parser source. Build before code mutation only after this plan compiles. Use the existing runner and identical hashed fixtures: baseline/new/JSON at 50k, both AST/DOM and common records, two adjacent rotated process rounds, ten samples, CPU 0. Smoke largest cases first; check small cases if a fast path changes. Preserve 180-second and 16-GiB guards. Investigate regressions above ten percent, retain failures/variance, profile speedups and roll back unhelpful hunks. Existing optional wrappers and item-list optimizations remain. Report exact source/binary provenance, raw samples, memory, SVG/PDF plot and residual JSON gap. CUDA remains conditional on the parser milestone; do not infer it from one case.

Load investigation, 2026-09-08 04:09 +02:00. CPU-0 campaign stopped after broad transient slowdowns and a 99-percent host load observation. Preserve completed raw cells as contaminated diagnostics; do not selectively keep favorable samples. External browser/download processes were observed and are not modified. Three follow-up per-CPU observations show CPU 2 idle while CPU 0 remains busy. Run a separate complete paired control on logical CPU 2 (affinity mask 4), all variants equally. Record the CPU-selection observations and inspect process-round spread before acceptance. This is still a shared desktop, not a claim of OS isolation. CPU-0 and CPU-2

timings stay separate.

Native Linux control, 2026-09-08 04:16 +02:00. The CPU-2 run also contains multi-fold drift; keep both Windows campaigns as diagnostics, not acceptance evidence. Amaterasu has low observed load, 32 GiB RAM and installed Rust 1.97.1. Additional ETA 15–20 minutes for transfer/build, native paired controls and report. Use archived before/after sources and the identical hashed fixtures, explicit cargo +1.97.1, matching release/debug flags, and no cross-platform ratio. Artifact directory /tmp/hymeko-parser-frontend-20260908; no new Git checkout or branch. Reuse Graph ProcessRunner for the Linux 50-ms VmHWM sampling and 180-second/16-GiB guards. Extend existing Parser Campaign with an optional checked invocation adapter; default Windows behavior unchanged. Additional files scripts/format_campaign.py and scripts/test_format_report.py; test adapter forwarding/resource rejection. No production dependency changes. The temporary Linux invocation only maps the existing monitor's output into the existing campaign schema. Preserve its exact source and Graph monitor hash. All five 50k inputs, both phases, baseline/new/JSON in adjacent reversed rounds; same CPU affinity for every variant. Candidate profile remains separately captured.

Owned-record diagnosis, 2026-09-08 12:00 +02:00. Owner: Parser benchmark adapter only; protected Core items empty. ETA 20 minutes: baseline extraction and staged diagnosis 6, ownership change and tests 5, matched native measurements 4, gates and report 5. Fresh fetch matched 284c490, zero divergence; preserve all existing work. Extend benches/formats/adapters.rs, benches/format_bench.rs, tests/format_comparison.rs and this plan/report bundle. Reuse Campaign's native invocation adapter, existing fixtures and Graph monitor. No dependencies, unsafe code, grammar or production API changes. Expose the existing benchmark Dataset projection for a pre-parsed-input measurement; retain its baseline source and executable before replacing borrowed traversal with consuming traversal and Cow::into_owned. Baseline diagnoses parse, borrowed projection and AST destruction separately. Candidate measures consuming conversion (including interleaved AST cleanup); do not compare that stage as pure projection. Both retain the identical full decode_records timing boundary and final-output destruction exclusion. Add dedicated drop_ast and consume_ast benchmark phases; baseline-only project_records is retained in the archived baseline, not as a duplicate production converter. Exercise new branches through Criterion smoke. Tests: exact records, negative zero, empty/nested/linked inputs, malformed record fields and ownership transfer of already-owned strings, including empty strings. Capture baseline CPU attribution if samply is available; keep prior profiler failure evidence explicit otherwise, and do not invent a hotspot share. Native Amaterasu: inspect load first, CPU 2, Rust 1.97.1, same source/fixture hashes, two reversed rounds, ten samples each. First stage smokes at 50k linked/long text; full decode A/B/JSON on five 50k patterns, focused small-case regression controls. 180-second process / observed 16-GiB cap; investigate greater-than-ten-percent regressions and uncertainty. Retain source snapshots under optimization/ownership; rollback only this stage's adapter and benchmark hunks if unhelpful. Record complete raw samples, IQR, peak RSS, source delta, checks and SVG/PDF plot.

Persistent regression checks, 2026-09-08 12:49 +02:00. ETA 20 minutes: gate and independent tests 9, native scale controls 4, gates and report/plot 7. Owner Parser; Core items empty; no dependency or algorithm changes. Fresh fetch matched origin/master 284c490, divergence zero. Extend existing scripts/format_report.py with a comparative gate and CLI, format_campaign.py with host identity, scripts/test_format_report.py with invalid-input, threshold, CLI and generated monotonicity/scale laws using already-installed Hypothesis. Extend tests/format_comparison.rs with nested owned/borrowed buffer checks. Before accepting, require same host/platform/machine/CPU affinity, phase boundary, fixture bytes/hash, matching complete two-round matrices, ten positive finite samples per round, successful process exits and valid observed resource budgets. Recompute statistics from raw samples; do not trust saved summary medians. Default time/peak regression

and round drift bounds ten percent. Distinguish pass, regression, invalid evidence and inconclusive drift; every non-pass exits nonzero. Do not mark noisy evidence green or manufacture absent historical host metadata. Expose a reusable local/CI command; no CI provider/runner is configured in this checkout, so do not claim an active remote pipeline. Mutation tests must prove the gate rejects a real slowdown and restored string copying fails the pointer test. Reuse immutable ownership before/after binaries and same native monitor; new 10k/50k five-pattern full decode controls plus matched JSON, two reversed rounds, production-scale smokes first. Preserve 180-second / observed 16-GiB guards, record CPU profiles already establishing current costs, and do not rerun unchanged Rust builds merely for measurements. Reuse existing plotter and report; store new raw evidence under optimization/regression-checks. Roll back only new hunks if gates fail; preserve all prior work. No new production parser or direct-AST bypass is attempted without its own semantic design and evidence.

